

Refactoring a Python Application with Dependency Injection (DI) and Inversion of Control (IoC)

Introduction

In object-oriented programming, tight coupling between classes can lead to inflexible, hard-to-test, and difficult-to-maintain code. The original `UserManager` class directly instantiates `EmailService`, violating key SOLID principles—particularly the Dependency Inversion Principle (DIP).

Dependency Injection (DI) and Inversion of Control (IoC) address this by decoupling high-level modules from low-level ones. Instead of a class creating its dependencies, they are provided externally (often via constructors, setters, or a DI container). This promotes flexibility, testability, and adherence to the Open-Closed Principle.

Problems in the Original Code

1. Tight Coupling: `UserManager` hard-codes `EmailService()`, making it impossible to swap notification methods (e.g., SMS, push notifications, or logging) without modifying the class.

2. Poor Testability: Unit tests for UserManager would trigger real email sends, which is slow, flaky, and requires external setup.

Original Tightly Coupled Code

python

```
class EmailService:
```

```
    def send_email(self, user, message):
```

```
        print(f"Sending email to {user}: {message}")
```

```
class UserManager:
```

```
    def __init__(self):
```

```
        self.notifier = EmailService() # Direct dependency - tight
```

coupling

```
    def register_user(self, user):
```

```
        self.notifier.send_email(user, "Welcome!")
```

Refactored Solution (Using DI and IoC)

We introduce an abstraction (NotificationService interface) that both concrete services implement. UserManager now depends on the abstraction, not the concrete class—following DIP.

Step-by-Step Refactoring

1. Define the Abstraction (Interface):

```
python
```

```
from abc import ABC, abstractmethod
```

```
class NotificationService(ABC):
```

```
    @abstractmethod
```

```
    def send_notification(self, user, message: str):
```

```
        """Send a notification to the user."""
```

```
        pass
```

2. Implement Concrete Services:

python

```
class EmailService(NotificationService):  
  
    def send_notification(self, user, message: str):  
  
        print(f"Sending email to {user}: {message}")
```

```
class SMSService(NotificationService):  
  
    def send_notification(self, user, message: str):  
  
        print(f"Sending SMS to {user}: {message}")
```

3. Inject the Dependency into UserManager:

python

```
class UserManager:  
  
    def __init__(self, notifier: NotificationService): # Dependency  
  
        injected via constructor  
  
        self.notifier = notifier
```

```
def register_user(self, user):  
  
    self.notifier.send_notification(user, "Welcome!")
```

Usage Example:

python

```
# Client code wires the dependencies (IoC in action)
```

```
email_service = EmailService()
```

```
user_manager = UserManager(email_service)
```

```
user_manager.register_user("alice@example.com")
```

```
sms_service = SMSService()
```

```
sms_manager = UserManager(sms_service)
```

```
sms_manager.register_user("+254712567890")
```

This design allows easy swapping of services without changing UserManager.

Demonstrating Benefits with Unit Tests

DI shines in testing. We can use mocks to isolate UserManager.

Example Unit Test (using unittest.mock):

```
python
```

```
import unittest
```

```
from unittest.mock import MagicMock
```

```
class TestUserManager(unittest.TestCase):
```

```
    def test_register_user_sends_welcome_notification(self):
```

```
        # Arrange: Create a mock notifier
```

```
        mock_notifier = MagicMock(spec=NotificationService)
```

```
        user_manager = UserManager(mock_notifier)
```

```
        # Act
```

```
user_manager.register_user("testuser")

# Assert: Verify interaction without real sending

mock_notifier.send_notification.assert_called_once_with("testuser",
"Welcome!")

if __name__ == "__main__":

    unittest.main()
```

It is noted that no emails are sent during tests—fast, reliable, and focused on the unit under test. This refactored design results in more maintainable, testable, and flexible code—core goals of modern software engineering.

References

Fowler, M. (2004) Patterns of Enterprise Application Architecture. Addison-Wesley

Martin, R.C. (2003) Agile Software Development: Principles, Patterns, and Practices.
Prentice Hall.

Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1995) Design Patterns:
Elements of Reusable Object-Oriented Software. Addison-Wesley.